# Java 入门笔记

第三课 · 输入输出、运算符与类型转换

| 学习目标 | 掌握 Scanner 输入、控制台输出、常见运算符、优先级以及基本类型之间的转换规则。 |
| --- | --- |

## 目录


本节目标 2

2.2.3 基本类型数据的输入与输出 2

2.2.4 var：局部变量类型推断 5

3.1 运算符 5

3.2 运算符优先级与结合性 10

3.3 表达式与类型转换 11

综合示例 13

常见易错点 13

课后自测 14

本课小结 14


整理日期：2026 年 9 月 22 日

> 本课承接 Class 2 的常量、变量和基本数据类型。重点是：会从键盘读取数据，会根据数据类型判断表达式的结果，并能正确使用常见运算符。

# 本节目标

- 掌握 Scanner 的基本使用方法，分清 next() 与 nextLine()。
- 理解 var 是局部变量的类型推断，变量仍然具有确定的静态类型。
- 掌握算术、关系、逻辑、赋值、条件和位运算符。
- 理解运算符的优先级、结合性与短路运算。
- 掌握基本类型的自动转换、强制转换和运算时的类型提升。

# 2.2.3 基本类型数据的输入与输出

## 一、使用 Scanner 读取键盘输入

Scanner 是 Java 标准库提供的输入工具，可以把输入内容解析为整数、小数、布尔值或字符串。它默认使用空白字符（空格、Tab、换行）分隔输入内容。

### 基本步骤

```
// 1. 导包：写在源文件顶部、class 声明之前
import java.util.Scanner;

public class ScannerDemo {
   public static void main(String[] args) {
      // 2. 创建 Scanner 对象
      Scanner sc = new Scanner(System.in);

      // 3. 调用方法读取数据
      System.out.print("请输入年龄：");
      int age = sc.nextInt();

      System.out.print("请输入身高：");
      double height = sc.nextDouble();

      System.out.print("是否为学生（true/false）：");
      boolean student = sc.nextBoolean();

      System.out.print("请输入姓名：");
      String name = sc.next();

      System.out.println("姓名：" + name);
      System.out.println("年龄：" + age);
      System.out.println("身高：" + height);
      System.out.println("是否为学生：" + student);

      // 4. 程序不再读取 System.in 时关闭
      sc.close();
   }
}
```

创建对象的一般形式是：

```
类名 变量名 = new 类名(构造参数);
Scanner sc = new Scanner(System.in);
```

其中 Scanner 是类型，sc 是引用变量，new Scanner(System.in) 创建对象，. 用来访问对象的方法。

### 常用读取方法

| 方法 | 返回类型 | 读取内容 |
| --- | --- | --- |
| nextByte() | byte | 一个 byte 范围的整数 |
| nextShort() | short | 一个 short 范围的整数 |
| nextInt() | int | 一个整数 |
| nextLong() | long | 一个长整数 |
| nextFloat() | float | 一个单精度浮点数 |
| nextDouble() | double | 一个双精度浮点数 |
| nextBoolean() | boolean | true 或 false，忽略大小写 |
| next() | String | 下一个由空白分隔的字符串 |
| nextLine() | String | 当前行剩余的全部内容 |

Scanner 没有 nextChar() 方法。读取单个字符时，先读取字符串，再取第一个字符：

```
char grade = sc.next().charAt(0);
```

如果输入为空字符串，charAt(0) 会越界；从控制台使用 next() 时，它会等待下一个非空 token，因此入门阶段通常可以这样写。完整方法列表见 Scanner 官方文档。

## 二、next() 与 nextLine() 的区别

假设输入：

```
Zhang San
```

- next() 只读取 Zhang，遇到空格停止。
- nextLine() 读取整行 Zhang San，但不包含末尾换行符。

一个常见陷阱是先读取数字，再立即读取整行文本：

```
System.out.print("请输入年龄：");
int age = sc.nextInt();

sc.nextLine(); // 吸收输入年龄后留下的换行符

System.out.print("请输入姓名：");
String name = sc.nextLine();
```

原因：nextInt() 读取了整数，却没有取走行末的换行符；后面的第一个 nextLine() 会先读到这段“空的行尾”。

如果整段输入以文本为主，也可以统一使用 nextLine()，再手动转换：

```
int age = Integer.parseInt(sc.nextLine());
double height = Double.parseDouble(sc.nextLine());
```

## 三、输入错误与简单校验

调用 nextInt() 时如果输入 abc，会发生 InputMismatchException。可以先判断输入是否符合要求：

```
System.out.print("请输入整数：");
if (sc.hasNextInt()) {
    int number = sc.nextInt();
    System.out.println("你输入了：" + number);
} else {
    String wrong = sc.next(); // 取走错误输入，避免一直停在原位置
    System.out.println("输入无效：" + wrong);
}
```

关闭 Scanner 会同时关闭它包装的输入源。sc.close() 若关闭了 System.in，本程序后面就不应再创建另一个 Scanner 继续读取键盘；通常只创建一个，在程序结束前关闭。

## 四、控制台输出

| 方法 | 特点 | 示例 |
| --- | --- | --- |
| print() | 输出后不自动换行 | System.out.print("A"); |
| println() | 输出后自动换行 | System.out.println("B"); |
| printf() | 按格式输出 | System.out.printf("%.2f%n", 3.14159); |

```
String name = "小明";
int age = 18;
double score = 89.5;

System.out.println("姓名：" + name + "，年龄：" + age);
System.out.printf("%s 的成绩是 %.1f%n", name, score);
```

常见格式占位符：

| 占位符 | 含义 |
| --- | --- |
| %d | 十进制整数 |
| %f | 浮点数；%.2f 表示保留两位小数进行格式化显示 |
| %s | 字符串或对象的文本表示 |
| %c | 字符 |
| %b | 布尔形式 |
| %n | 与当前系统匹配的换行 |

+ 两边只要有一边是字符串，通常会进行字符串拼接。运算顺序会影响结果：

```
System.out.println(1 + 2 + "Java"); // 3Java
System.out.println("Java" + 1 + 2); // Java12
System.out.println("Java" + (1 + 2)); // Java3
```

## 2.2.4 var：局部变量类型推断

Java 10 起，可以在某些局部变量声明中使用 var。类型由编译器根据初始值推断，不是 JVM 在运行时猜测，var 也不是“任意类型”。

```
var age = 18;        // 推断为 int
var price = 9.9;     // 推断为 double
var name = "小明";    // 推断为 String
var sc = new Scanner(System.in); // 推断为 Scanner

age = 20;          // 正确：仍然是 int
// age = "二十";     // 错误：不能把 String 赋给 int
```

### 使用限制

- 只能用于局部变量，包括部分 for 循环变量和资源变量。
- 声明时必须有初始值：不能写 var value;。
- 不能直接用 null 推断：不能写 var value = null;。
- 不能用作字段、方法参数、构造方法参数或方法返回类型。
- 一次声明一个变量，不能写 var a = 1, b = 2;。
- var 只是省略重复、明显的类型名称；类型不明显时，显式写出类型通常更易读。

```
// 以下写法都不合法：
// var count;
// var data = null;
// var x = 1, y = 2;
// public var getName() { ... }
```

规则见 Java 语言规范：局部变量类型推断。

# 3.1 运算符

## 3.1.0 操作数、运算符和表达式

```
表达式：a + b * 2
操作数：a、b、2
运算符：+、*
```

表达式由操作数和运算符组成，执行后会得到一个值。判断表达式结果时，依次考虑：

1. 运算符的优先级和结合性；
2. 操作数的数据类型；
3. 是否发生类型提升或类型转换；
4. 是否有副作用，例如 i++ 会修改变量。

## 3.1.1 算术运算符

| 运算符 | 含义 | 示例 | 结果 |
| --- | --- | --- | --- |
| + | 加法或字符串拼接 | 3 + 4 | 7 |
| - | 减法 | 3 - 4 | -1 |
| * | 乘法 | 3 * 4 | 12 |
| / | 除法 | 1 / 2 | 0 |
| % | 求余 | 20 % 3 | 2 |

### 整数除法与浮点除法

```
System.out.println(1 / 2);       // 0：两个操作数都是整数
System.out.println(1.0 / 2.0);   // 0.5
System.out.println((double) 1 / 2); // 0.5
```

整数除法直接舍去小数部分，方向是趋近于 0：-7 / 2 的结果为 -3。

整数除以 0 会抛出 ArithmeticException；浮点数除以 0.0 通常得到 Infinity、-Infinity 或 NaN。

### 余数的符号

```
 11 % 2 =  1
 11 %-2 =  1
-11 % 2 = -1
-11 %-2 = -1
```

Java 中余数的符号与被除数（左操作数）一致，与除数符号无关；余数绝对值小于除数绝对值。规范定义见 JLS 15.17.3。

### 整数溢出

```
int max = Integer.MAX_VALUE;
System.out.println(max + 1); // -2147483648
```

普通整数运算溢出时不会自动变为 long，通常也不会抛异常。需要更大范围时，应在运算前使用 long：

```
int a = 100_000;
int b = 100_000;
long wrong = a * b;       // int 乘法先溢出，再赋给 long
long right = (long) a * b;  // 先把一个操作数转为 long
```

## 3.1.2 自增、自减与一元运算符

++ 和 -- 的操作数必须是可修改的数值变量，不能直接作用于常量或普通计算结果。

```
int x = 5;
int a = ++x; // 先把 x 加 1，再取值：x=6，a=6

int y = 5;
int b = y++; // 先取 y 的旧值，再加 1：b=5，y=6
```

| 写法 | 含义 |
| --- | --- |
| ++x | 先自增，再使用新值 |
| x++ | 先使用旧值，再自增 |
| --x | 先自减，再使用新值 |
| x-- | 先使用旧值，再自减 |

如果只是单独一行 x++; 或 ++x;，执行后效果相同。复杂表达式中连续使用同一个变量的 ++、-- 很难阅读，应拆成多行。

一元 + 表示正值，一元 - 表示取相反数：

```
int n = 5;
System.out.println(-n); // -5
```

### 3.1.3 赋值运算符

简单赋值的形式：

```
变量 = 表达式;
```

左侧必须是可赋值的变量、数组元素或字段，不能是字面量或普通表达式。

```
int x;
x = 10;

int a, b;
a = b = 5; // 赋值从右向左结合，a 和 b 都得到 5
```

#### 复合赋值运算符

+=、-=、*=、/=、%=、&=、|=、^=、<<=、>>=、>>>=

```
int x = 10;
x += 3;     // 相当于 x = x + 3，结果为 13
x *= 2 + 1;  // 相当于 x = x * (2 + 1)，结果为 39
```

复合赋值会把运算结果隐式转换回左侧变量的类型，因此并非在所有细节上都等同于简单展开：

```
byte n = 10;
n += 1;          // 正确：结果隐式转换回 byte
// n = n + 1;     // 错误：n + 1 的结果是 int
n = (byte) (n + 1); // 显式转换后正确
```

规范中的准确等价形式是 E1 op= E2 约等于 E1 = (T) (E1 op E2)，并且左侧只求值一次。参见 JLS 15.26.2。

不要混淆：= 表示赋值，== 表示比较是否相等。

### 3.1.4 关系运算符与相等运算符

| 运算符 | 含义 |
| --- | --- |
| >、>= | 大于、大于等于 |
| <、<= | 小于、小于等于 |
| ==、!= | 等于、不等于 |

比较结果一定是 boolean 值：true 或 false。

```
int x = 8;
int y = 5;
System.out.println(x + y >= 10); // true
System.out.println(x == y);     // false
```

数学中的 a < x <= b 不能原样写进 Java，必须拆成两个比较：

```
boolean inRange = a < x && x <= b;
```

String 等引用类型使用 == 比较时，比较的是两个引用是否指向同一个对象；比较字符串内容通常使用 equals()：

```
String first = new String("Java");
String second = new String("Java");

System.out.println(first == second);     // false：不是同一对象
System.out.println(first.equals(second)); // true：内容相同
```

浮点数还需注意 NaN：Double.NaN == Double.NaN 的结果是 false。

### 3.1.5 逻辑运算符

逻辑运算符的操作数是布尔表达式：

| 运算符 | 含义 | 示例 |
| --- | --- | --- |
| ! | 逻辑非 | !true 为 false |
| && | 短路与 | 两边都为 true 才为 true |
| \|\| | 短路或 | 至少一边为 true 就为 true |
| & | 非短路与（布尔操作数） | 两边都会计算 |
| \| | 非短路或（布尔操作数） | 两边都会计算 |
| ^ | 逻辑异或（布尔操作数） | 两边不同时为 true |

常用优先级：! 高于 &&，&& 高于 ||。不确定时加括号。

```
boolean result = age >= 18 && hasTicket;
boolean allowed = isMember || score >= 90;
```

短路运算

- A && B：如果 A 为 false，结果已经确定，不再计算 B。
- A || B：如果 A 为 true，结果已经确定，不再计算 B。

```
int divisor = 0;
boolean safe = divisor != 0 && 10 / divisor > 1;
System.out.println(safe); // false，不会执行除以 0
```

&、| 用于布尔值时不短路，两边都会计算。一般条件判断优先使用 &&、||。短路规则见 JLS 15.23-15.24。

### 3.1.6 条件运算符

条件运算符 ?: 是 Java 唯一的三目运算符：

```
条件 ? 条件为 true 时的表达式 : 条件为 false 时的表达式
```

```
int score = 75;
String result = score >= 60 ? "及格" : "不及格";
System.out.println(result);
```

它会产生一个值，适合简单的二选一。分支中有多条语句或逻辑复杂时，使用 if-else 更清楚。

### 3.1.7 位运算符

位运算主要针对整数类型的二进制位。

| 运算符 | 名称 | 规则 |
| --- | --- | --- |
| & | 按位与 | 两位都是 1 才为 1 |
| \| | 按位或 | 至少一位为 1 就为 1 |
| ^ | 按位异或 | 两位不同时为 1 |
| ~ | 按位取反 | 0 变 1，1 变 0 |
| << | 左移 | 左移，右侧补 0 |
| >> | 有符号右移 | 右移，左侧用符号位填充 |
| >>> | 无符号右移 | 右移，左侧补 0 |

#### &, |, ^, ~ 示例

```
a = 102 = 0110 0110
b =  59 = 0011 1011

a & b   = 0010 0010 = 34
a | b   = 0111 1111 = 127
a ^ b   = 0101 1101 = 93
```

Java 的整数使用二进制补码。对 int 来说：

```
System.out.println(~5); // -6，因为 ~x == -x - 1
```

### 移位示例

```
int value = 12; // 二进制 0000...1100
System.out.println(value << 2);  // 48
System.out.println(value >> 2);  // 3

int negative = -8;
System.out.println(negative >> 1);  // -4，左侧补 1
System.out.println(negative >>> 1); // 2147483644，左侧补 0
```

对不发生溢出的非负整数，a << n 可以理解为 a * $2^n$，a >> n 可以类比除以 $2^n$。但负数的取整方式、溢出和高位丢失会使这种类比失效，所以移位的本质仍是二进制位移动。

补充：int 的实际移位距离只使用右操作数的低 5 位，long 使用低 6 位。例如 1 << 32 实际等于 1 << 0。初学时避免使用大于等于类型位数的移位距离。

## 3.1.8 其他常见运算符

| 运算符 | 作用 | 示例 |
| --- | --- | --- |
| new | 创建对象或数组 | new Scanner(System.in)、new int[5] |
| instanceof | 判断对象是否属于某种类型 | obj instanceof String |
| () | 改变运算顺序、调用方法、书写参数列表 | (a + b) * c、println(x) |
| [] | 声明数组类型或访问数组元素 | int[] data、data[0] |
| . | 访问对象或类的成员 | sc.nextInt()、System.out |
| (□□) | 强制类型转换 | (double) 1 / 3 |

# 3.2 运算符优先级与结合性

优先级决定“先算哪一种运算符”；结合性只在优先级相同时决定组合方向。它们不等于运行时所有子表达式的求值顺序。

从高到低记住下面的常用层级即可：

| 优先级 | 运算符 | 常见结合方向 |
| --- | --- | --- |
| 高 | ()、[]、.、后缀 x++、x-- | 从左到右 |
|  | 一元 ++x、--x、+、-、!、~、强制转换 | 从右到左 |
|  | *、/、% | 从左到右 |
|  | +、- | 从左到右 |
|  | <<、>>、>>> | 从左到右 |
|  | <、<=、>、>=、instanceof | 从左到右 |
|  | ==、!= | 从左到右 |
|  | &，然后 ^，然后 \| | 从左到右 |
|  | &&，然后 \|\| | 从左到右 |

| 优先级 | 运算符 | 常见结合方向 |
| --- | --- | --- |
|  | ?: | 从右到左 |
| 低 | =、+=、-= 等赋值运算符 | 从右到左 |

```
int value = 2 + 3 * 4;    // 14
int grouped = (2 + 3) * 4; // 20
```

实际写代码时，不要依赖读者背完整优先级表。只要括号能让意图更清楚，就应使用括号。

# 3.3 表达式与类型转换

## 3.3.1 自动类型转换（扩大转换）

数值范围较窄的类型在一定条件下可自动转为较宽的类型：

```
byte → short → int → long → float → double
       char → int → long → float → double
```

这张图用于帮助记忆，但要注意：byte、short、char 之间不能互相任意自动转换；char 是无符号 16 位类型，也不能自动转为 short。

```
byte small = 10;
int count = small;    // byte → int
long total = count;   // int → long
double result = total; // long → double
```

扩大转换通常不会丢失数值的数量级，但不保证总能保持精确值。例如很大的 int 转成 float、很大的 long 转成 double 可能损失末尾精度。完整规则见 JLS 5.1.2。

## 3.3.2 运算时的类型提升

byte、short、char 参与普通算术运算时，通常会先提升为 int：

```
byte a = 10;
byte b = 20;
// byte sum = a + b;     // 错误：a + b 的类型是 int
int sum = a + b;         // 正确
byte smallSum = (byte) (a + b); // 确认范围安全后可强转
```

两个数值操作数进行运算时，大致按以下顺序提升：

1. 有一个是 double，另一个转为 double；
2. 否则有一个是 float，另一个转为 float；
3. 否则有一个是 long，另一个转为 long；
4. 否则两边都提升为 int。

```
System.out.println(5 / 2);    // int 运算，结果 2
System.out.println(5 / 2.0);  // double 运算，结果 2.5
System.out.println(5L + 2);   // long 运算，结果 7L
```

### 3.3.3 强制类型转换（缩小转换）

格式：

```
(目标类型) 表达式
```

```
double value = 3.9;
int number = (int) value; // 3：直接截去小数部分，不是四舍五入

int large = 130;
byte small = (byte) large; // -126：超出 byte 范围后高位被截去
```

强制转换告诉编译器“我接受可能的信息损失”，并不保证结果符合直觉。整数之间的窄化可能溢出，浮点数转整数会向 0 截断。

boolean 与任何数值类型之间都不能直接转换。

### 3.3.4 让 1 / 3 得到小数

```
System.out.println(1 / 3);       // 0
System.out.println((double) 1 / 3); // 0.3333333333333333
System.out.println(1.0 / 3);     // 0.3333333333333333
```

(double) 1 / 3 的执行过程：

1. 把左操作数 1 转为 double 值 1.0；
2. 右操作数 3 自动转为 double 值 3.0；
3. 执行浮点除法。

不要写 (double) (1 / 3) 来期待小数，因为括号内已经先完成整数除法并得到 0，之后只能把 0 转成 0.0。

# 综合示例

```
import java.util.Scanner;

public class ScoreCalculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("请输入姓名：");
        String name = sc.nextLine();

        System.out.print("请输入三门成绩（用空格分隔）：");
        double first = sc.nextDouble();
        double second = sc.nextDouble();
        double third = sc.nextDouble();

        double average = (first + second + third) / 3;
        boolean passed = average >= 60;
        String level = average >= 90 ? "优秀" : "继续努力";

        System.out.printf("%s 的平均分：%.2f%n", name, average);
        System.out.println("是否及格：" + passed);
        System.out.println("评价：" + level);

        sc.close();
    }
}
```

示例输入：

```
小明
95 88 92
```

输出：

```
小明 的平均分：91.67
是否及格：true
评价：优秀
```

# 常见易错点

1. Scanner 没有 nextChar()；读取字符可用 next().charAt(0)。
2. nextInt() 后直接调用 nextLine()，常会先读到遗留的换行符。
3. var 是编译器的局部变量类型推断，不是动态类型，也不能只写声明而不给初始值。
4. 20 % 3 的结果是 2；余数符号跟随被除数。
5. 1 / 2 是整数除法，结果为 0；让任一操作数成为浮点数才能得到 0.5。
6. 前置 ++x 先修改再取值，后置 x++ 先取旧值再修改。
7. = 是赋值，== 是相等比较；字符串内容通常使用 equals() 比较。
8. && 和 || 会短路，& 和 | 用于布尔值时两边都会计算。
9. byte、short、char 做普通算术运算时通常先提升为 int。
10. 强制转换可能丢失精度、截去小数或改变数值。

11 整数溢出通常不会报错；把结果赋给 long 不等于运算过程自动使用 long。
12 复杂表达式不要只靠优先级，应使用括号并拆分步骤。

# 课后自测

1 next() 与 nextLine() 有什么区别？
2 为什么 Scanner 不能写 sc.nextChar()？怎样读取一个字符？
3 var value = 3.14; 中 value 的实际类型是什么？之后能赋字符串吗？
4 11 % -2 和 -11 % 2 分别是多少？
5 执行 int x = 5; int y = x++; 后，x、y 分别是多少？
6 为什么 byte c = a + b; 通常不能通过编译，即使 a、b 都是 byte？
7 false && 10 / 0 > 1 为什么不会发生除以 0？
8 String result = score >= 60 ? "□□" : "□□□"; 使用了什么运算符？
9 (double) (1 / 3) 与 (double) 1 / 3 的结果有何区别？
10 long result = 100_000 * 100_000; 为什么可能先得到错误结果？

## 参考答案

1 next() 读取下一个由空白分隔的 token；nextLine() 读取当前行剩余内容。
2 Scanner 没有该方法；可写 sc.next().charAt(0)。
3 double；不能赋字符串，因为推断完成后类型固定。
4 1 和 -1，余数符号跟随左侧被除数。
5 x 为 6，y 为 5。
6 byte 参与普通算术运算时会提升为 int，a + b 的结果类型是 int。
7 && 短路；左侧为 false 时不会计算右侧。
8 条件运算符，也叫三目运算符。
9 前者先做整数除法，结果是 0.0；后者先把 1 转为 double，结果约为 0.3333333333333333。
10 两个操作数都是 int，乘法先按 int 运算并溢出，之后才把错误结果转成 long；应写 (long) 100_000 * 100_000 或使用 100_000L。

# 本课小结

- Scanner 根据所调用的方法把输入解析为对应类型；混用 token 读取与整行读取时要处理换行符。
- var 省略的是局部变量声明中的类型名称，编译后变量仍有确定类型。
- 表达式结果同时受运算符规则、优先级和操作数类型影响。
- &&、|| 的短路特性能避免不必要或危险的右侧计算。
- 数值运算会发生类型提升；缩小范围时需要强制转换，并承担数据丢失风险。